

Buenas Prácticas para el Desarrollo de Aplicaciones Reactivas

DANIEL ALEXANDER MIRANDA CASTILLO

Universidad Mariano Galvez

SEMINARIO DE TECNOLOGÍAS DE INFORMACIÓN

Mélvín Cali

01 de agosto de 2026

La primera vez que me topé con el tema de aplicaciones reactivas fue revisando un proyecto que manejaba varias peticiones al mismo tiempo, y ahí entendí por qué tanto se habla de esto hoy en día: cuando un sistema tiene que responder rápido a eventos que pueden venir de cualquier lado (un usuario, otro servicio, un flujo de datos que no para), la forma tradicional de programar ya no da abasto.

De lo que he leído y de lo poco que he trabajado con esto, hay ciertas prácticas que me parecen las más importantes:

1. **Tener claro el Reactive Manifesto desde el diseño.** No es solo teoría bonita: si desde el inicio no se piensa en que el sistema sea responsivo, resiliente, elástico y basado en mensajes, después es un dolor de cabeza tratar de meterlo con calzador.
2. **No ignorar el backpressure.** Esto para mí es lo que más se pasa por alto. Si un componente genera datos más rápido de lo que otro puede procesar, y no hay un plan (buffer, descartar, limitar), tarde o temprano el sistema se satura y todo se cae.
3. **Evitar bloquear hilos.** Usar I/O no bloqueante (como WebFlux en Java o RxJS en JS) suena obvio pero en la práctica es fácil meter una operación bloqueante sin darse cuenta, y eso arruina todo el beneficio de ser reactivo.
4. **No dejar los errores sueltos.** En un flujo reactivo el error también es parte del flujo, así que hay que manejarlo con operadores como `onErrorResume` o `retry` en vez de dejar que explote en algún punto random.
5. **Poder ver qué está pasando (observabilidad).** Como todo es asíncrono y a veces distribuido, sin métricas o trazabilidad en tiempo real es casi imposible saber dónde se rompió algo cuando falla.

Ahora, si me preguntan cuál es la más importante, para mí es sin duda el manejo del back-pressure. Le prestamos mucha atención al manejo de errores y a la observabilidad porque son más “visibles”, pero de lo que he visto, la mayoría de proyectos reactivos que fallan en producción no

fallan por eso, sino porque nadie pensó qué pasaba cuando un componente no daba abasto con la cantidad de datos.

¿Ustedes qué opinan, están de acuerdo o creen que hay otra práctica que pesa más que el backpressure?